supervisor 컴포넌트

개요

supervisor는 Worldcoin Orb 시스템의 중앙 조정 서비스로, 독립적인 에이전트들(바이너리/프로세스) 간의 디바이스 상태와 외부 UX를 조정하는 핵심 IPC 서버입니다. 이 컴포넌트는 높은 권한 수준(root)에서 실행되며, 시스템의 전반적인 건강성과 일관된 사용자 경험을 보장하는 역할을 담당합니다.

supervisor는 분산된 Orb 시스템 아키텍처에서 중앙 허브 역할을 하며, 각 컴포넌트가 독립적으로 동작하면서도 전체 시스템이 조화롭게 작동할 수 있도록 조정합니다.

설계 원칙

1. 관심사의 분리 (Separation of Concerns)

- 각 컴포넌트의 책임을 명확히 분리
- 복잡한 모놀리식 애플리케이션 대신 전용 서비스 활용
- 공격 표면 최소화를 위한 권한 제한

2. 디바이스 건강성 관리

- 열 관리 및 팬 제어
- 업데이트 프로세스 조정
- 시스템 리소스 모니터링

3. 일관된 사용자 경험

- 모든 시스템 상태에서 통일된 피드백 제공

- 업데이트 중에도 운영자에게 상태 정보 전달
- LED 및 사운드를 통한 시각적/청각적 피드백

주요 기능

1. 프로세스 간 조정 (IPC)

- gRPC over Unix Domain Socket을 통한 통신
- D-Bus를 통한 시스템 서비스 통신
- 컴포넌트 간 상태 동기화

2. 시스템 생명주기 관리

- 안전한 시스템 종료 및 재부팅
- 컴포넌트별 우아한 종료 처리
- 업데이트 프로세스 조정

3. 하드웨어 제어

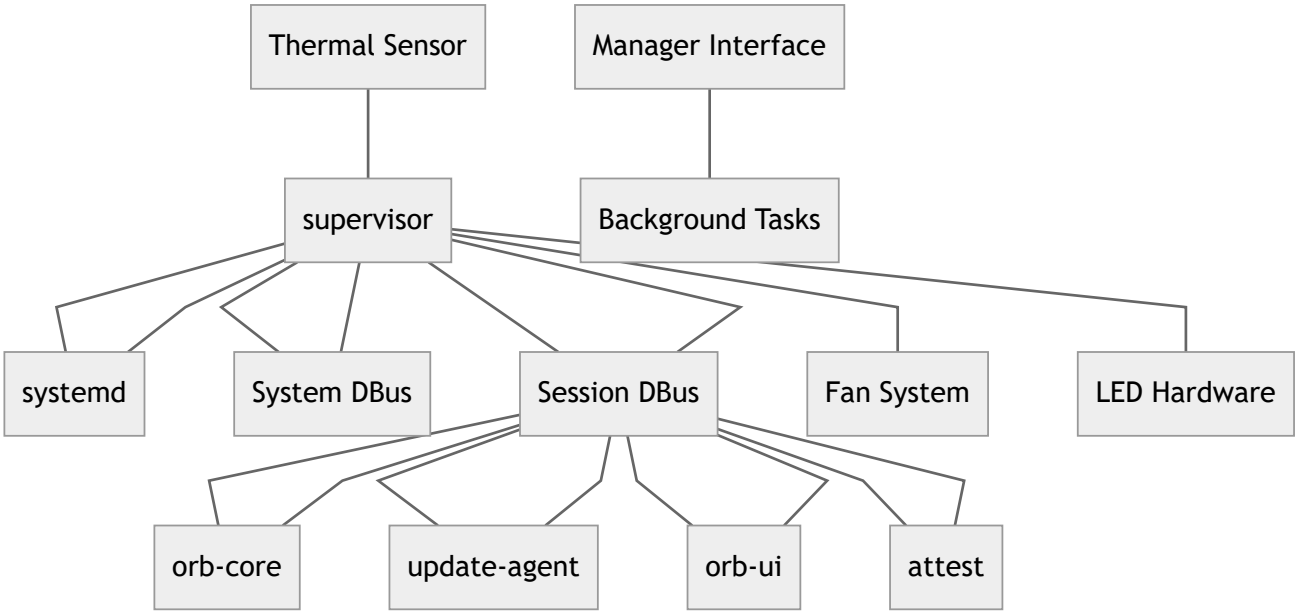
- 팬 속도 제어 및 열 관리
- LED 패턴 제어
- 사운드 시스템 관리

4. 보안 및 권한 관리

- 높은 권한이 필요한 작업의 중앙 집중화
- 다른 컴포넌트의 권한 최소화
- 시스템 보안 경계 유지

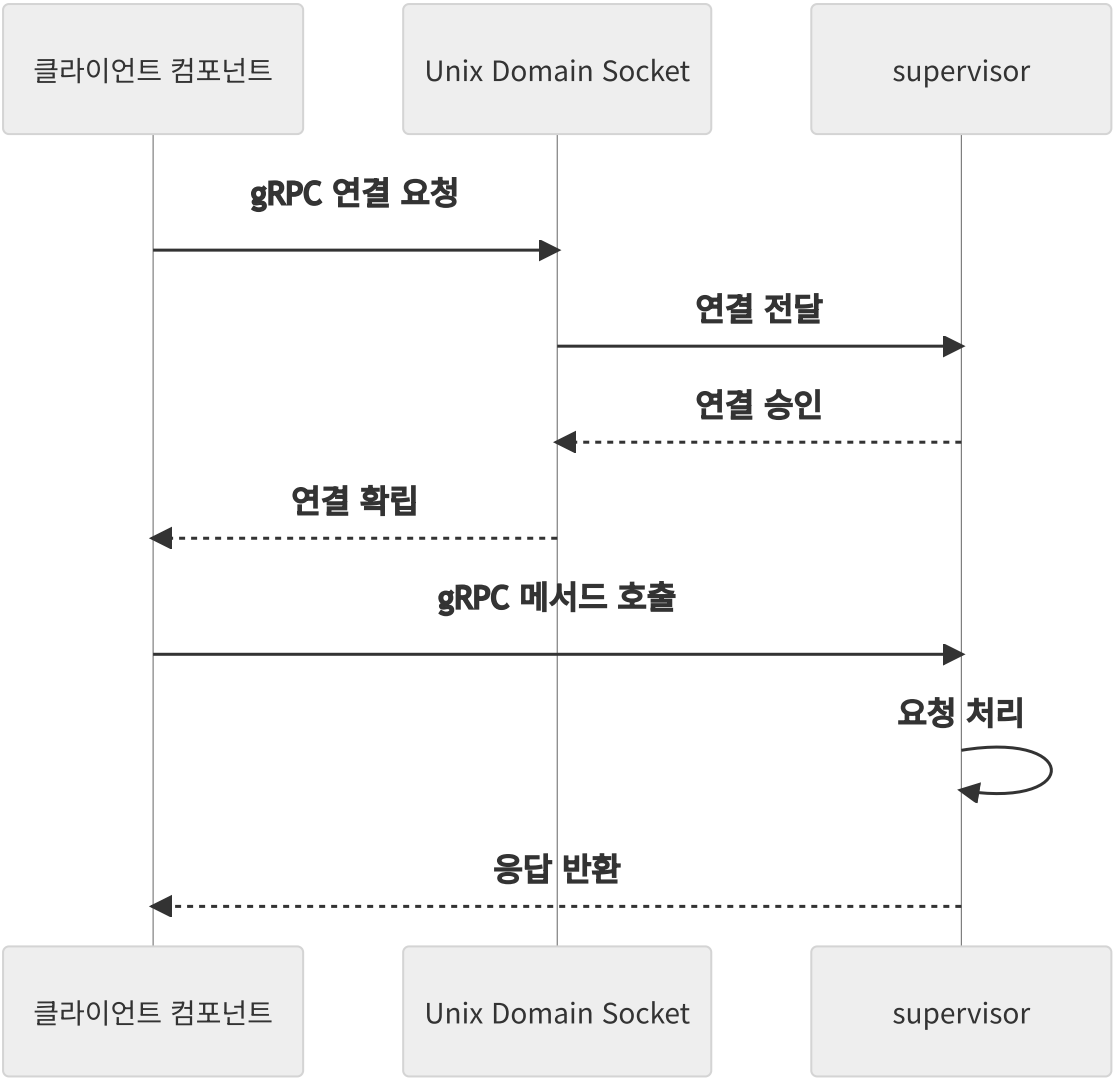
아키텍처

시스템 구조

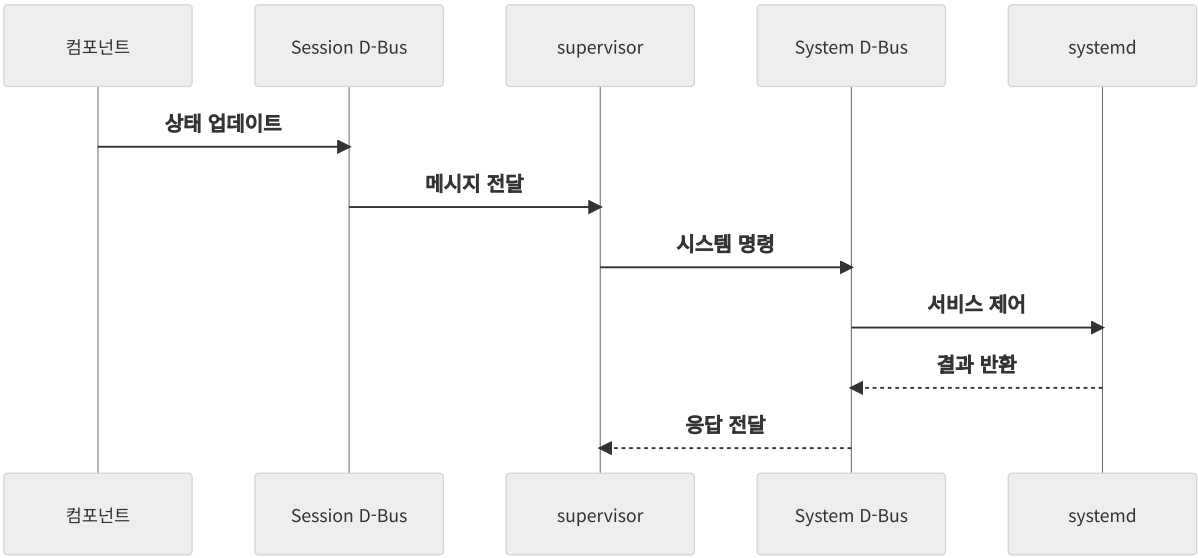


통신 프로토콜

gRPC over Unix Domain Socket


클라이언트 컴포넌트
Unix Domain Socket
supervisor
gRPC 연결 요청
연결 전달
연결 승인
연결 확립
gRPC 메서드 호출
요청 처리
응답 반환
클라이언트 컴포넌트
Unix Domain Socket
supervisor

D-Bus 통신



디바이스 상태 관리

1. 헬스 모니터링

supervisor는 시스템의 전반적인 건강성을 지속적으로 모니터링합니다:

온도 관리

```
// 온도 모니터링 및 팬 제어 예시
async fn monitor_thermal_state() -> Result<()> {
let mut thermal_monitor = ThermalMonitor::new();
loop {
let temperature = thermal_monitor.read_temperature().await?;
let fan_speed = calculate_fan_speed(temperature);
```

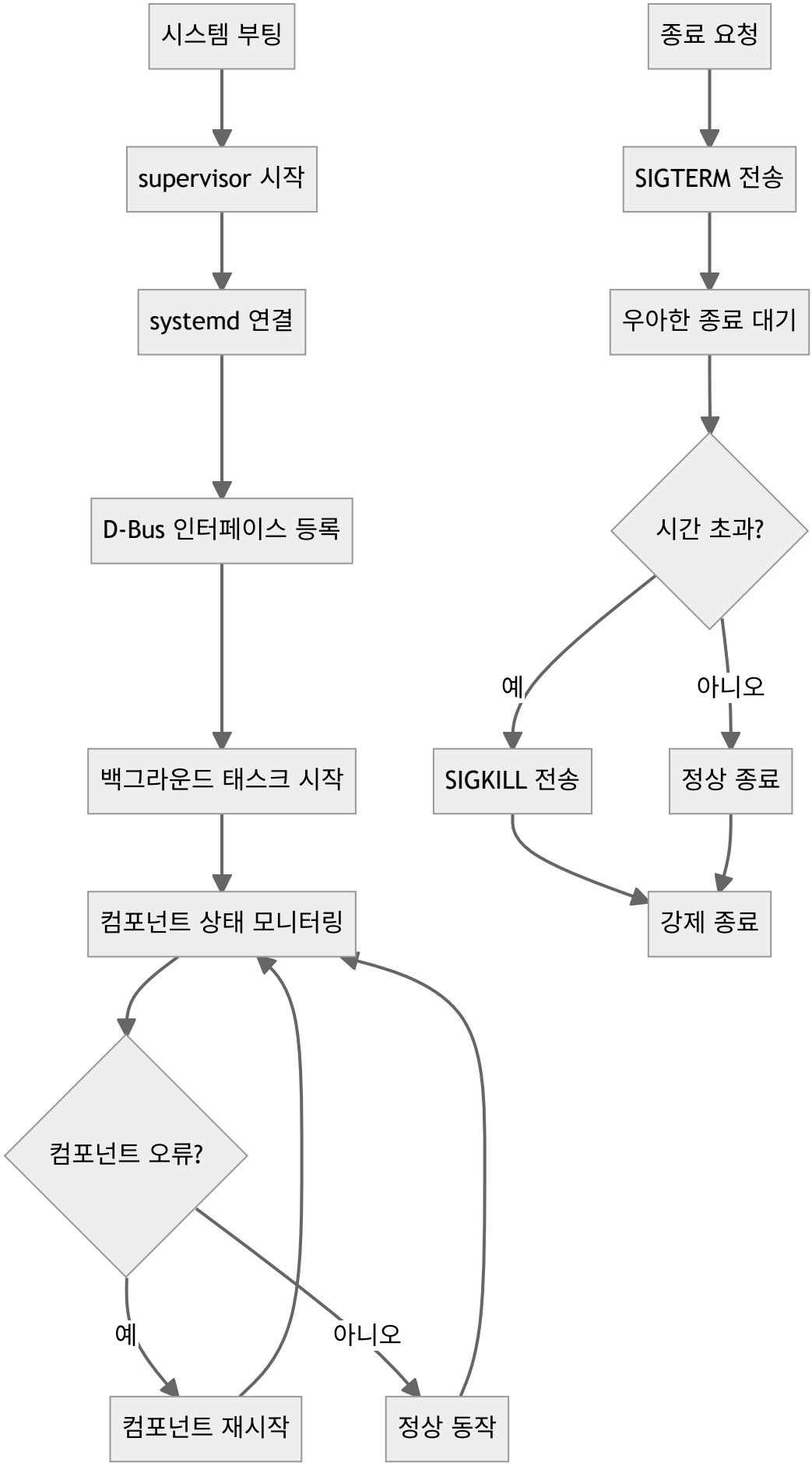
```
if temperature > CRITICAL_TEMPERATURE {
warn!("Critical temperature reached: {}°C", temperature);
emergency_shutdown().await?;
} else if temperature > WARNING_TEMPERATURE {
warn!("High temperature detected: {}°C", temperature);
set_fan_speed(fan_speed).await?;
}
tokio::time::sleep(Duration::from_secs(5)).await;
}
}
```

리소스 사용량 추적

- CPU 사용률 모니터링
- 메모리 사용량 추적
- 디스크 I/O 및 SSD 온도 감시
- 네트워크 연결 상태 확인

2. 프로세스 생명주기 관리

컴포넌트 시작 및 중지


시스템 부팅
supervisor 시작
systemd 연결
D-Bus 인터페이스 등록
백그라운드 태스크 시작
컴포넌트 상태 모니터링
컴포넌트 오류?
예
아니오
컴포넌트 재시작
정상 동작
종료 요청
SIGTERM 전송
우아한 종료 대기
시간 초과?
예
아니오
SIGKILL 전송
정상 종료
강제 종료

종료 프로세스

```
// 우아한 종료 처리
async fn graceful_shutdown(components: Vec<Component>) -> Result<()> {
info!("Starting graceful shutdown sequence");
// 1단계: 모든 컴포넌트에 SIGTERM 전송
for component in &components {
if let Some(pid) = component.get_pid() {
signal::kill(pid, Signal::SIGTERM)?;
info!("Sent SIGTERM to {}", component.name());
}
}
// 2단계: 정의된 시간 동안 대기
let shutdown_timeout = Duration::from_secs(30);
let deadline = Instant::now() + shutdown_timeout;
while Instant::now() < deadline {
let mut all_stopped = true;
for component in &components {
if component.is_running() {
all_stopped = false;
break;
}
}
if all_stopped {
```

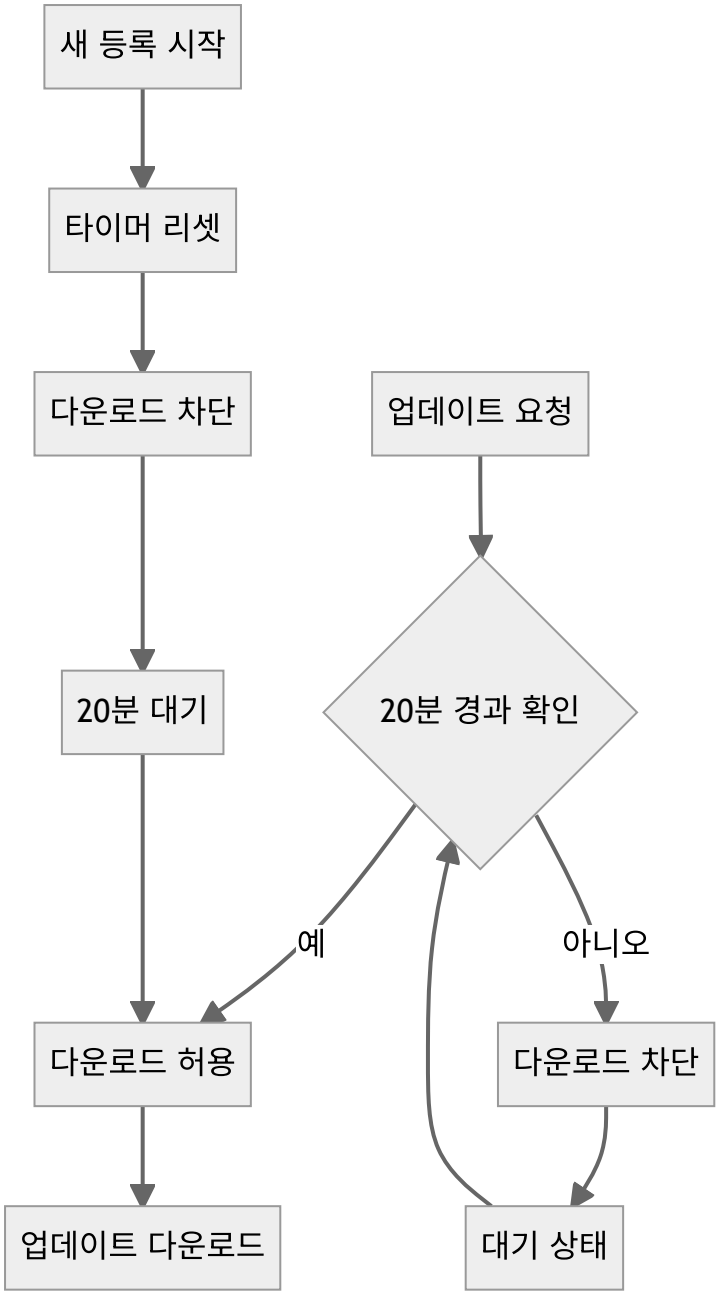
```
info!("All components shut down gracefully");
return Ok(());
}
tokio::time::sleep(Duration::from_millis(500)).await;
}
// 3단계: 여전히 실행 중인 컴포넌트에 SIGKILL 전송
warn!("Timeout reached, sending SIGKILL to remaining processes");
for component in &components {
if component.is_running() {
if let Some(pid) = component.get_pid() {
signal::kill(pid, Signal::SIGKILL)?;
warn!("Sent SIGKILL to {}", component.name());
}
}
}
Ok(())
}
```

업데이트 조정

1. 백그라운드 다운로드 관리

supervisor는 업데이트 에이전트의 다운로드 권한을 관리합니다:

다운로드 허용 로직

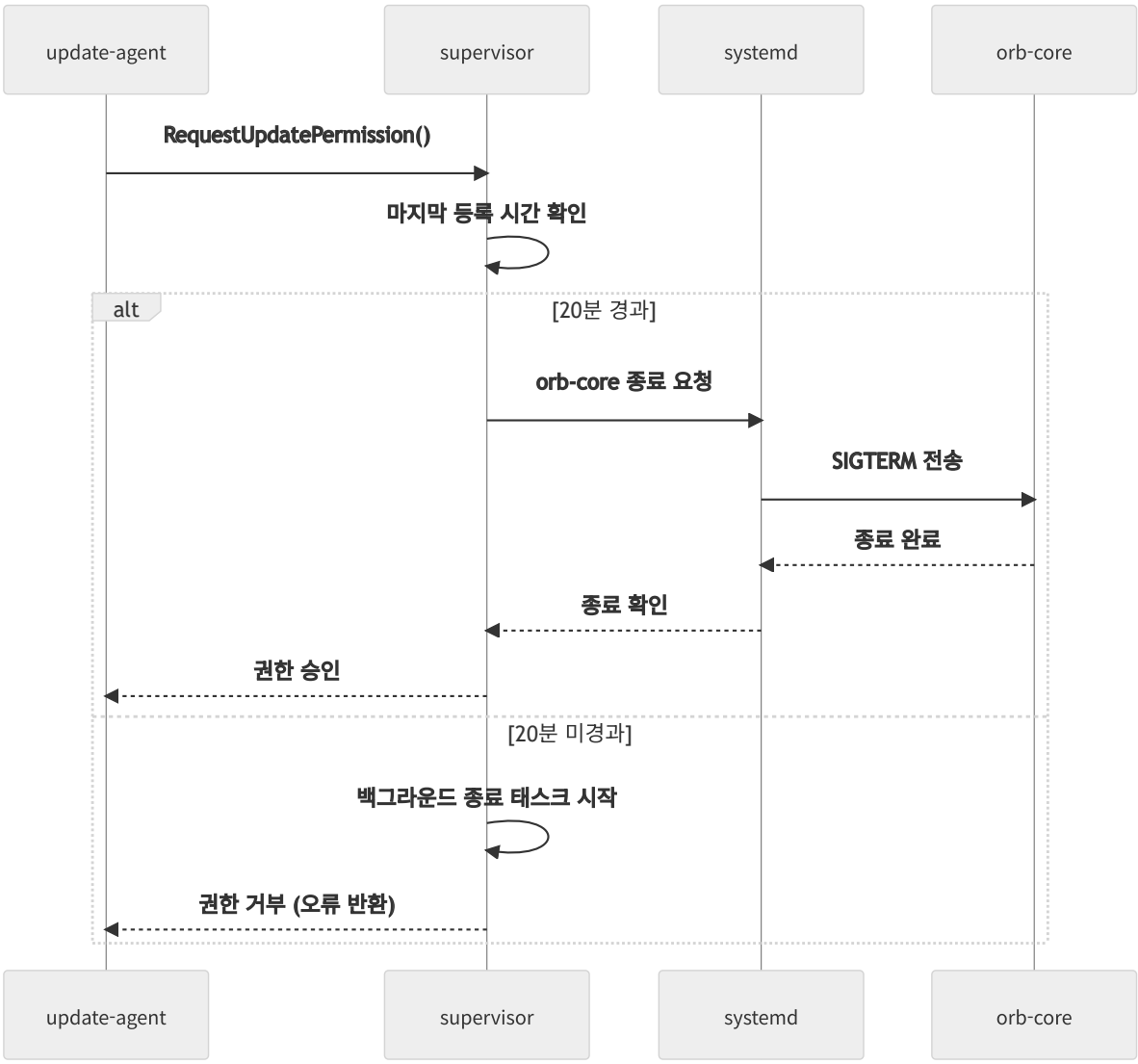


BackgroundDownloadsAllowed 속성

```
#[zbus(property, name = "BackgroundDownloadsAllowed")]
async fn background_downloads_allowed(&self) -> bool {
let elapsed = self.last_signup_event.borrow().elapsed();
elapsed >= self.duration_to_allow_downloads
}
```

2. 업데이트 권한 요청 처리

RequestUpdatePermission 메서드



3. 컴포넌트 종료 및 업데이트 시작

```
async fn request_update_permission(&self) -> Result<(), BusError> {
```

```
let  conn = self.system_connection.as_ref()
.expect("manager must be connected to system dbus");
let  systemd_proxy = systemd1::ManagerProxy::new(conn).await?;
// orb-core 종료 태스크 시작
let  mut  shutdown_core_task =
spawn_shutdown_worldcoin_core_timer(
systemd_proxy.clone(),
self.last_signup_event.subscribe(),
);
// 1초 대기하여 즉시 종료 가능한지 확인
match tokio::time::timeout(Duration::from_secs(1), &mut  shutdown_core_task).awa
Ok(Ok(Ok(()))) => {
debug!("orb-core가 1초 내에 종료됨, 업데이트 허용");
Ok(())
}
Err(_) => {
debug!("orb-core 종료에 1초 이상 소요, 백그라운드에서 처리");
// 백그라운드에서 종료 후 업데이트 에이전트 시작
spawn_start_update_agent_after_core_shutdown_task(
systemd_proxy,
shutdown_core_task,
);
Err(BusError::updates_blocked(
```

```
"orb core가 여전히 실행 중이며 마지막 등록 후 20분 후에 종료됩니다"
))
}
}
}
```

시스템 종료 및 재부팅 관리

1. 예약된 종료

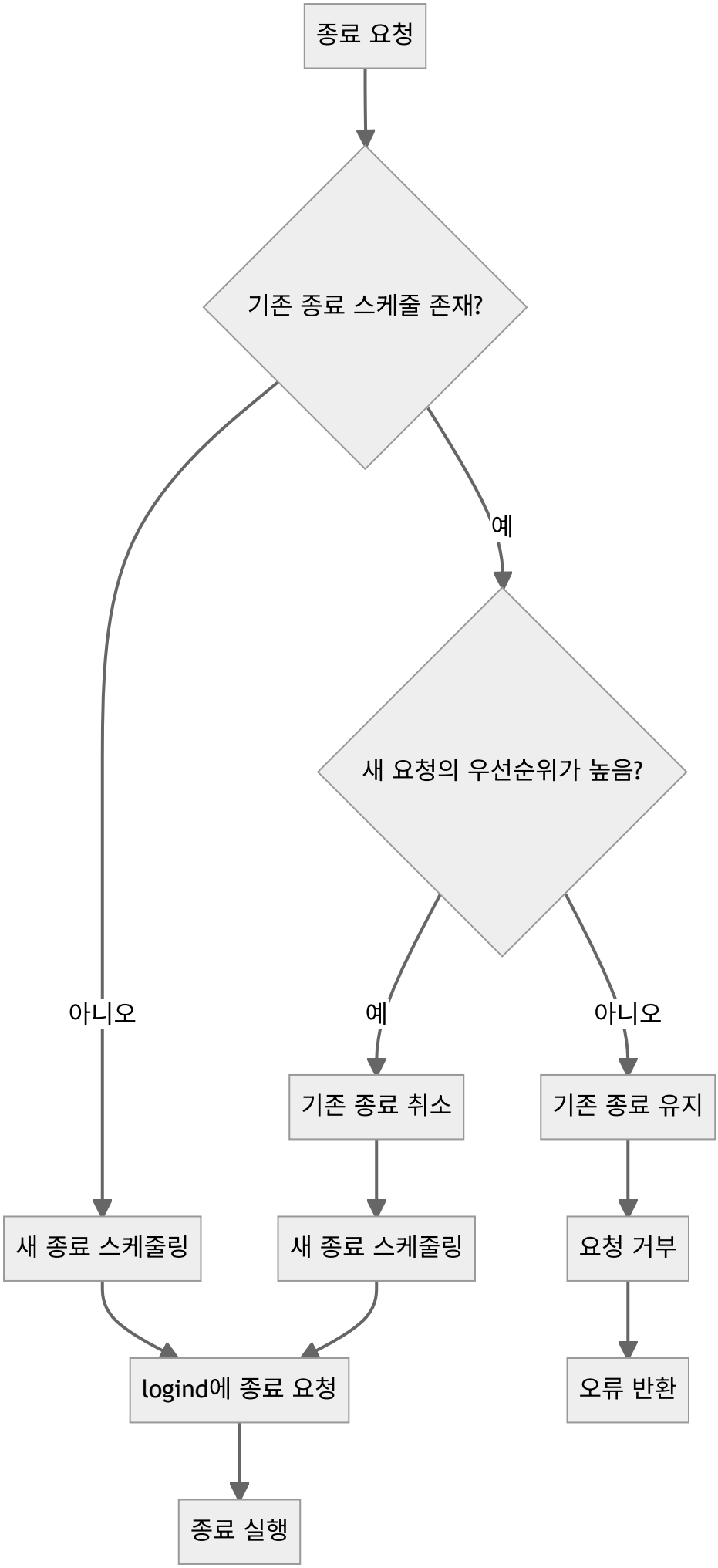
supervisor는 시스템의 안전한 종료를 관리합니다:

ScheduleShutdown 메서드

```
async fn schedule_shutdown(&self, kind: &str, when: u64) -> zbus::fdo::Result<
let shutdown_request = ScheduledShutdown::try_from_dbus((kind.to_owned(), when)
.ok_or(FdoError::InvalidArgs("empty string".to_owned()))?;
let conn = self.system_connection.as_ref()
.expect("manager must be connected to system dbus");
let logind_proxy = login1::ManagerProxy::new(conn).await?;
let preemption_info = schedule_shutdown(logind_proxy, shutdown_request.clone())
match preemption_info {
PreemptionInfo::NoExistingShutdown => {
info!("예약된 종료 스케줄링: {:?}", shutdown_request);
}
```

```
PreemptionInfo::PreemptedExistingShutdown(s) => {
warn!("기존 낮은 우선순위 종료 {:?}를 새로운 종료 {:?}로 대체", s, shutdown_request);
}
PreemptionInfo::KeptExistingShutdown(s) => {
warn!("높은 우선순위 기존 종료 {:?}로 인해 새로운 종료 {:?} 스킵", s, shutdown_request
}
}
Ok(())
}
```

2. 종료 우선순위 관리


종료 요청
기존 종료 스케줄 존재?
예
아니오
새 요청의 우선순위가 높음?
예
아니오
기존 종료 취소
기존 종료 유지
새 종료 스케줄링
새 종료 스케줄링
요청 거부
logind에 종료 요청
오류 반환
종료 실행

UX 조정 (LED, 사운드)

1. LED 제어 시스템

supervisor는 시스템 전반의 LED 패턴을 조정합니다:

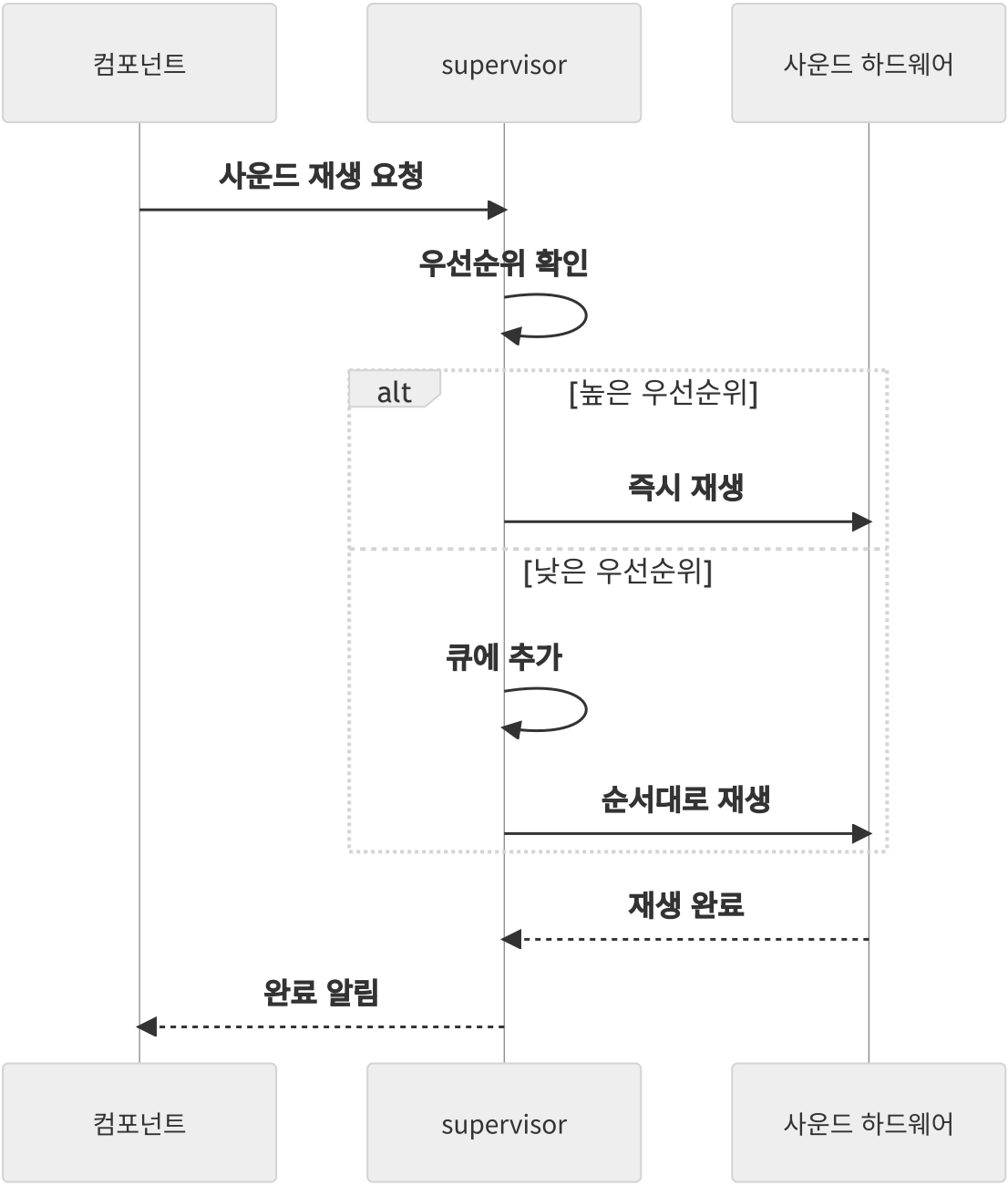
LED 패턴 관리

```
// LED 상태 조정
async fn coordinate_led_patterns() -> Result<()> {
let mut led_controller = LedController::new();
let mut system_state = SystemState::Idle;
loop {
match system_state {
SystemState::Updating => {
led_controller.set_pattern(LedPattern::UpdateInProgress).await?;
}
SystemState::SignupInProgress => {
// orb-core가 LED 제어를 담당
led_controller.delegate_to_component("orb-core").await?;
}
SystemState::Error(error_type) => {
led_controller.set_pattern(LedPattern::Error(error_type)).await?;
}
SystemState::Idle => {
```

```
led_controller.set_pattern(LedPattern::Idle).await?;

}

}

tokio::time::sleep(Duration::from_millis(100)).await;

}

}
```

2. 사운드 시스템 관리

시스템 사운드 조정



팬 제어 및 열 관리

1. 온도 기반 팬 제어

```
struct ThermalManager {
temperature_sensors: Vec<TemperatureSensor>,
fan_controllers: Vec<FanController>,
thermal_zones: Vec<ThermalZone>,
```

```
}

impl ThermalManager {
async fn manage_thermal_state(&mut self) -> Result<()> {
loop {
let max_temp = self.read_max_temperature().await?;
let fan_speed = self.calculate_fan_speed(max_temp);
match max_temp {
temp if temp > EMERGENCY_TEMP => {
error!("Emergency temperature reached: {}°C", temp);
self.emergency_shutdown().await?;
}
temp if temp > CRITICAL_TEMP => {
warn!("Critical temperature: {}°C", temp);
self.set_fan_speed(100).await?; // 최대 속도
}
temp if temp > WARNING_TEMP => {
info!("High temperature: {}°C", temp);
self.set_fan_speed(fan_speed).await?;
}
_ => {
self.set_fan_speed(fan_speed).await?;
}
```

```
}
tokio::time::sleep(Duration::from_secs(2)).await;
}
}
fn calculate_fan_speed(&self, temperature: f32) -> u8 {
// 온도에 따른 팬 속도 계산 (0-100%)
let base_temp = 40.0; // 기준 온도
let max_temp = 80.0; // 최대 온도
if temperature <= base_temp {
20 // 최소 팬 속도
} else if temperature >= max_temp {
100 // 최대 팬 속도
} else {
let ratio = (temperature - base_temp) / (max_temp - base_temp);
(20.0 + ratio * 80.0) as u8
}
}
}
```

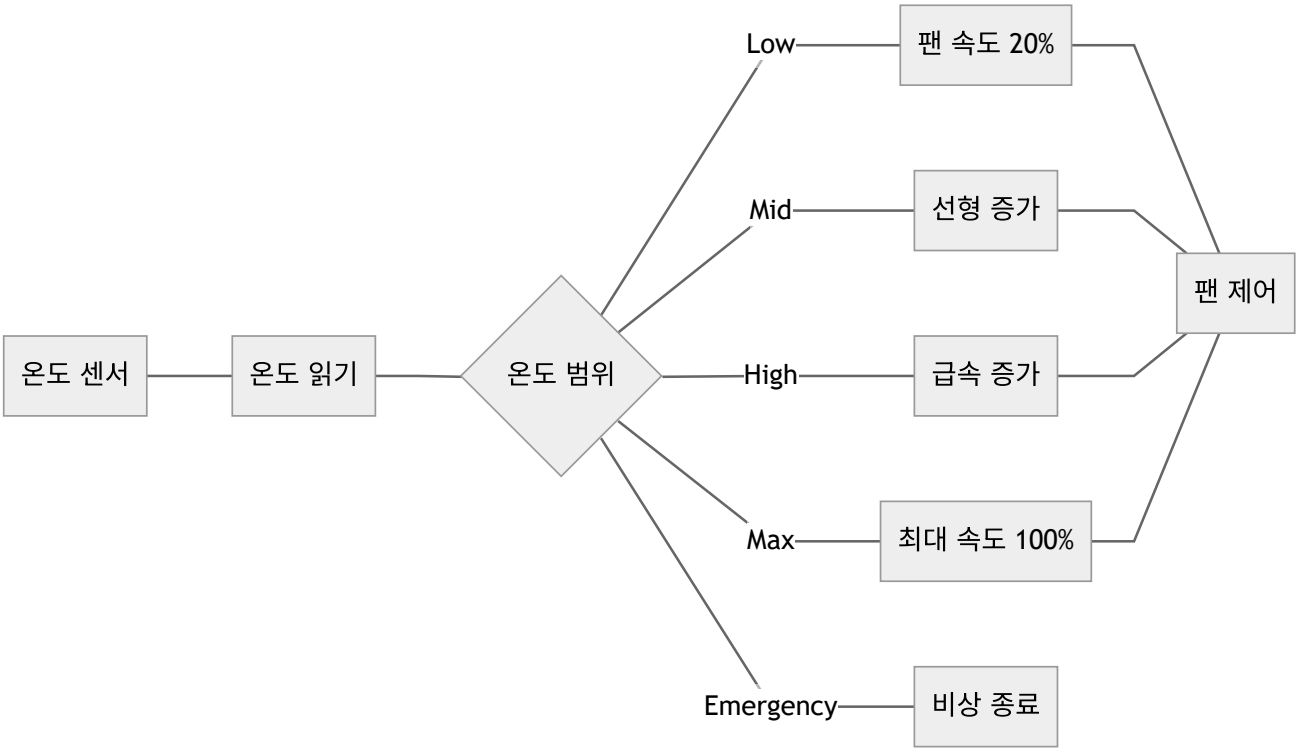
2. 열 관리 정책

온도 임계값

- **정상 동작**: < 50°C

- **경고 수준**: 50°C - 70°C
- **위험 수준**: 70°C - 85°C
- **비상 종료**: > 85°C

팬 제어 전략



D-Bus 인터페이스

org.worldcoin.OrbSupervisor1.Manager

속성 (Properties)

- `BackgroundDownloadsAllowed` : 백그라운드 다운로드 허용 여부

메서드 (Methods)

- `RequestUpdatePermission()` : 업데이트 권한 요청
- `ScheduleShutdown(kind: string, when: uint64)` : 예약된 종료

시그널 (Signals)

- `PropertiesChanged` : 속성 변경 알림

사용 예시

D-Bus 명령어를 통한 재부팅 예약

```
# 10초 후 재부팅 예약
busctl call \
--address=unix:path=/tmp/worldcoin_bus_socket \
org.worldcoin.OrbSupervisor1 \
/org/worldcoin/OrbSupervisor1/Manager \
org.worldcoin.OrbSupervisor1.Manager \
ScheduleShutdown \
st "reboot" $(date -d "+10 seconds" +%s%N | cut -c1-16)
```

설정 및 구성

주요 설정

```
[supervisor]
# D-Bus 연결 설정
session_dbus_path = "/run/user/1000/bus"
system_dbus_path = "/run/dbus/system_bus_socket"
```

```
# 인터페이스 경로
manager_object_path = "/org/worldcoin/OrbSupervisor1/Manager"
well_known_name = "org.worldcoin.OrbSupervisor1"


# 타이밍 설정
duration_to_allow_downloads = "20m"  # 다운로드 허용까지 대기 시간
shutdown_timeout = "30s"  # 컴포넌트 종료 대기 시간


[thermal]
# 온도 임계값 (섭씨)
warning_temperature = 50.0
critical_temperature = 70.0
emergency_temperature = 85.0


# 팬 제어
min_fan_speed = 20  # 최소 팬 속도 (%)
max_fan_speed = 100  # 최대 팬 속도 (%)
```

로깅 및 모니터링

supervisor는 구조화된 로깅을 통해 시스템 상태를 추적합니다:

```
use tracing::{info, warn, error, instrument};
```

```
#[instrument(skip(self))]
async fn handle_signup_started(&mut self) -> Result<()> {
info!("Signup started event received");
// 다운로드 타이머 리셋
self.reset_last_signup_event();
// 속성 변경 알림
self.notify_property_changed("BackgroundDownloadsAllowed").await?;
info!("Background downloads blocked for {} minutes",
self.duration_to_allow_downloads.as_secs() / 60);
Ok(())
}
```

로그 카테고리

- **시스템 이벤트**: 시작, 종료, 재부팅
- **컴포넌트 상태**: 프로세스 시작/종료, 오류
- **열 관리**: 온도 변화, 팬 속도 조정
- **업데이트 조정**: 권한 요청, 다운로드 상태
- **D-Bus 통신**: 메서드 호출, 시그널 전송

오류 처리 및 복구

1. 컴포넌트 오류 처리

```
async fn handle_component_failure(component: &str, error: &Error) -> Result<()
```

```
error!("Component {} failed: {}", component, error);
match  component {
"orb-core" => {
// 중요한 컴포넌트이므로 즉시 재시작
restart_component("orb-core").await?;
}
"update-agent" => {
// 업데이트 에이전트는 필요시에만 실행
warn!("Update agent failed, will restart on next update");
}
"orb-ui" => {
// UI 컴포넌트 재시작
restart_component("orb-ui").await?;
}
_ => {
warn!("Unknown component failed: {}", component);
}
}
Ok(())
}
```

2. 시스템 복구 전략

자동 복구 메커니즘

- 컴포넌트 자동 재시작
- 네트워크 연결 복구
- 하드웨어 상태 재초기화
- 설정 파일 복구

수동 개입이 필요한 상황

- 하드웨어 오류
- 파일 시스템 손상
- 보안 침해 감지
- 복구 불가능한 시스템 오류

이 문서는 supervisor 컴포넌트의 핵심 기능과 시스템 내에서의 역할을 설명합니다. supervisor는 Orb 시스템의 안정성과 일관성을 보장하는 중요한 컴포넌트로, 지속적인 모니터링과 유지보수가 필요합니다.